Debugging Python Code on Codility

https://gemini.google.com/app/54b0d0dad699da93

User prompt: I need some help debugging code for my CISC 121 assignment. When i put my code into codility and press run I am met with this error: Compilation successful. Example test: [9, 3, 9, 3, 9, 7, 9] Output (stderr): Traceback (most recent call last): File "/tmp/exec_user_yhx6qkpt/exec.py", line 115, in <module> main() File "/tmp/exec_user_yhx6qkpt/exec.py", line 70, in main sol = __import__('solution') ^^^^^^^^^^^^^^^^^^^^^ File "/tmp/exec_user_yhx6qkpt/solution.py", line 6 for num in A: IndentationError: unexpected indent RUNTIME ERROR (tested program terminated with exit code 1) Detected some errors. What does this error mean. Draft is below: def solution(A): missing_pair= {} FOR num in A: if num in missing_pair: missing_pair.remove(num) else: missing_pair.add(num) return missing_pair.pop()

Response: The error `IndentationError: unexpected indent` means Python encountered leading whitespace (tabs or spaces) where it wasn't expecting an indented block, or there is an inconsistent mix of tabs and spaces across the lines.

Beyond the indentation, your draft has three other syntax and logic issues that will cause immediate runtime errors once the indentation is fixed:

1. **`FOR` is capitalized:** Python keywords are case-sensitive. It must be lowercase `for`.
2. **`missing_pair = {}` creates an empty dictionary, not a set:** In Python, `{}` defaults to a `dict`. Dictionaries do not have `.add()` or `.remove()` methods (sets do), which will trigger an `AttributeError`. Use `set()` instead.
3. **Indentation mixing:** When copying and pasting into Codility's web editor, tabs (`\t`) and spaces often get mixed up. Using 4 standard spaces per indent level avoids this completely.

Corrected Code

```
def solution(A):
    missing_pair = set()
    for num in A:
        if num in missing_pair:
            missing_pair.remove(num)
        else:
            missing_pair.add(num)
    return missing_pair.pop()
```

Complexity & Alternative Note

- **Current Set Approach:** Runs in $O(N)$ time and takes up to $O(N)$ extra memory space.
- **Optimal Bitwise Approach ($O(1)$ Space):** Since every element except one appears in pairs, and $x \oplus x = 0$, you can solve this with constant memory using the XOR (^) operator:

```
def solution(A):
    unpaired = 0
    for num in A:
        unpaired ^= num
    return unpaired
```
